

RESUELTOS

SISTEMAS DISTRIBUIDOS I [75.74]

Exámen Integrador



[Francisco O. Lorda](#)

105554

github.com/gcc-florda

[Carolina Di Matteo](#)

103963

[github/gcc-cdimatteo](https://github.com/gcc-cdimatteo)

1. ¿Qué diferencia una arquitectura Multithreading de una Multiprocessing? Indique ventajas.

- en multi-threading la comunicación es sencilla y es fácil compartir recursos (se comparte: heap, data segment, code segment, file descriptors); a diferencia de multi-processing que la comunicación es a través de IPC y es difícil compartir recursos entre procesos (solo comparten code segment)
- los sistemas que están hechos sobre arquitecturas multi-processing son más escalables y estables que los que están hechos sobre arquitecturas multi-threading ya que estos están pensados para correr en un sólo equipo y por eso es fácil compartir recursos entre ellos
- los programas multi-threading tienen un alto acoplamiento entre recursos, a diferencia de los multi-processing, por esta razón son fáciles de escalar

VENTAJAS MULTI-THREADING

- + útil para programas simples con un objetivo común donde se aprovecha la herramienta de dividir una tarea en tareas más pequeñas; aún más, estas tareas pueden ser paralelizables
- + fácil comunicación (memoria compartida)
- + fácil compartir recursos/datos (se comparte el heap, code segment, data segment, file descriptors)
- + implementación sencilla

VENTAJAS MULTI-PROCESSING

- + fácil escalabilidad para aquellos programas que deseen ser distribuidos en varios equipos
- + bajo acoplamiento entre componentes (lo que facilita la distribución)
- + un proceso fallido no afecta al resto de procesos (no existen deadlocks)

2. Explique cómo garantizar un corte consistente al capturar un estado global de cierto sistema distribuido. Indique los pasos principales del algoritmo propuesto.

Historia → conjunto de eventos de un proceso.

Corte → conjunto de historias de todos los procesos de un sistema hasta cierto evento k .

Un corte es consistente si por cada evento que contiene, está el evento que ocurre antes que él.

Se puede proponer el algoritmo de Chandy & Lamport para obtener un snapshot del estado global de un sistema distribuido. Los pasos del algoritmo son:

- 1) El proceso observador guarda su estado
- 2) El proceso observador envía un marcador (snapshot request message) a todos los procesos
- 3) El proceso que recibe un marcador por primera vez
 - a) envía al proceso observador su propio estado
 - b) adjunta el marcador a todos los mensajes que va a mandar luego para propagar el marcador
- 4) Si un proceso que ya recibió un marcador recibe un mensaje que NO contiene un marcador, significa que este mensaje fue creado antes de que se reciba un marcador; por lo tanto, como tiene que formar parte del snapshot, el proceso reenvía el mensaje al observador

3. Explique qué estrategias puede implementar un MOM para garantizar tolerancia a fallos. Ejemplifique con algún MOM que conozca o haya implementado.

Estrategias para garantizar tolerancia a fallos:

- ACK de mensajes para asegurar su entrega
- Replicación de colas
- Durabilidad de colas (almacenamiento durable de mensajes; en lugar de que sólo existan en memoria, bajarlos a disco ante posibles caídas)

En el TP usamos RabbitMQ para la comunicación entre workers en el sistema distribuido.

4. Explique Distributed Shared Memory (DSM) desarrollando al menos una alternativa de implementación. Indique ventajas y desventajas de dicha arquitectura.

DSM es un IPC que da la ilusión de una memoria distribuida entre varios procesos/clientes.

- + Procesos no necesitan conocerse pues en todos los enfoques se comunican con un proceso orquestador que se encarga de proveerles la memoria solicitada
- + Intuitivo para el desarrollo de sistemas distribuidos
- Single Point of Failure → si se cae el proceso orquestador, a priori no existe otro que se encargue de administrar el almacenamiento
- Latencia → en el mejor de los casos ningún proceso tiene que esperar por memoria y la latencia sólo está compuesta por el tardío de enviar un mensaje al proceso orquestador que se encarga de prestar memoria y la entrega de memoria efectivamente; ahora bien, los diferentes enfoques de DSM tienen como objetivo disminuir la latencia que puede surgir de la concurrencia al acceso del almacenamiento compartido

REPLICACIÓN DE PÁGINAS (sólo lectura)

El servidor tiene la memoria paginada. Los distintos clientes/procesos piden al servidor el acceso de lectura a una página; éste les da la réplica de la página solicitada. Cuando un cliente/proceso quiere escribir sobre la réplica de una página, lo hace y notifica al servidor. Este, modifica la página original en su equipo y anula aquellas réplicas de los clientes/procesos restantes que no conozcan de esta actualización.

5. Describa al menos 2 patrones de carga de aplicaciones Web e indique cómo el escalamiento horizontal permite mitigar efectos negativos.

Predictable Burst → la carga aumenta de manera repentina pero este incremento es predecible

Unpredictable Burst → la carga del sistema aumenta de manera impredecible

En ambos casos si la infraestructura del sistema no está preparada para manejar las altas demandas, podría generar una baja en el sistema.

El escalamiento horizontal permite repartir las requests de los clientes con un load balancer entre los distintos equipos de procesamiento del sistema. En caso de que aún así suceda alguna baja, la disponibilidad del sistema ya no depende de un único recurso si no que hay otros que respaldan la respuesta a pedidos de clientes.

6. Indique las diferencias entre los modelos de Replicación Activa y Pasiva. Indique las ventajas de cada uno.

En la Replicación Activa múltiples nodos procesan las requests recibidas. Todas las respuestas son enviadas al Load Balancer o bien al Middleware quien se encarga de brindar una única respuesta al cliente. Ante la baja de algún nodo, simplemente existen otras réplicas que realizan la misma tarea que el caído. La caída de una réplica no implica que el tiempo de respuesta aumente.

En la Replicación Pasiva sólo un nodo procesa las requests recibidas y entrega las respuestas correspondientes a la entidad que se encarga de devolver dicha respuesta al Cliente. Las réplicas existen a modo de “back up” ante la baja del nodo que se encuentra activo contestando las requests. Los nodos “back up” controlan que el nodo activo no se haya caído efectivamente, en pos de brindar disponibilidad al cliente final. El nodo activo actualiza las réplicas. Las réplicas no tienen que realizar cómputo redundante.

7. Explique un algoritmo de consenso para que varios procesos puedan llegar a un acuerdo sobre cierta variable que ellos definen. Indique cuántas réplicas son necesarias para garantizar que el algoritmo funcione si a) se cae 1 proceso, b) se caen 2 procesos.

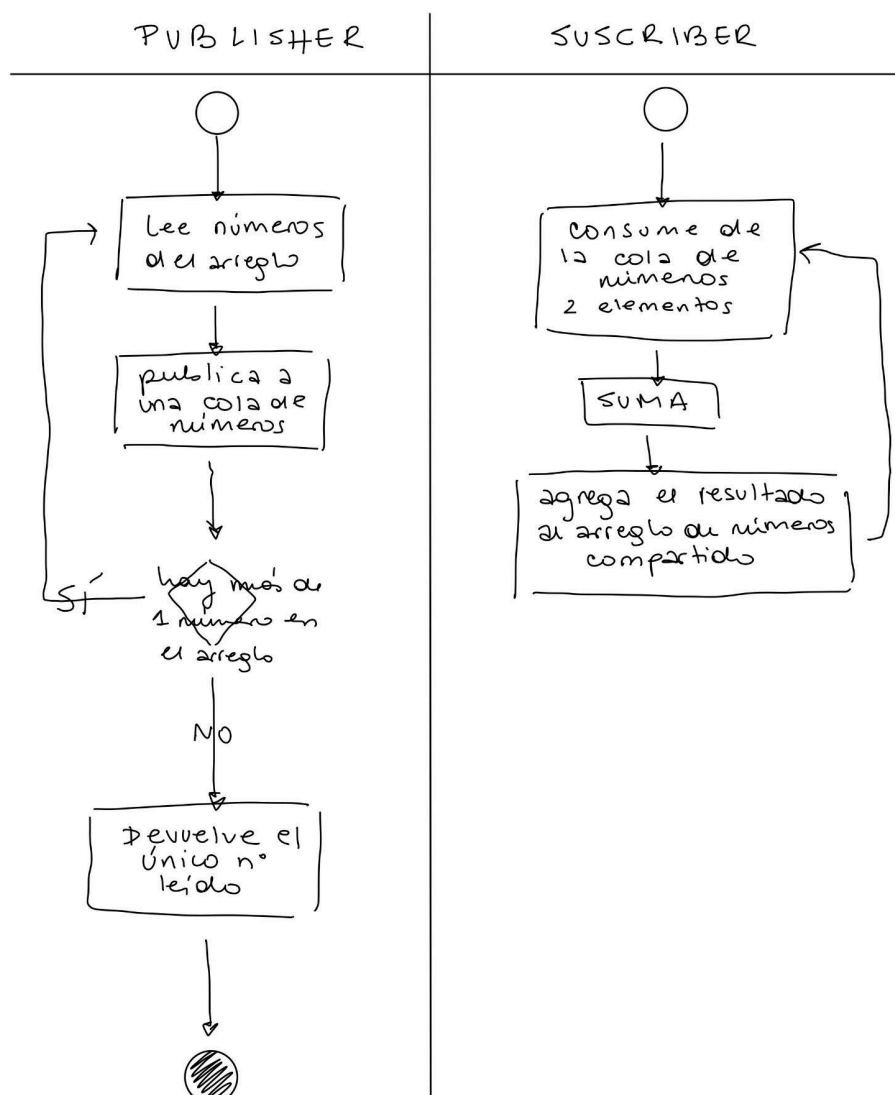
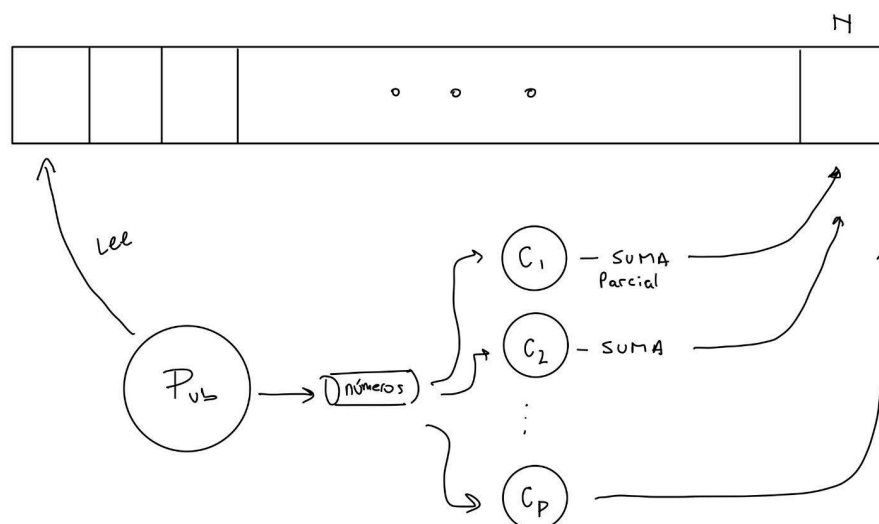
Paxos es un algoritmo de consenso que se utiliza para que varios procesos lleguen a un acuerdo sobre un valor. Los pasos son:

1. Cliente envía un request(v) que es recibido por el Proposer
2. Proposer prepara una propuesta con un ID "N", y envía a los Acceptors $prepare(N)$
3. Acceptors prometen aceptar la propuesta con el ID más grande que hayan recibido hasta el momento (prometen rechazar cualquier propuesta con un ID menor al que prometen); y envían al proposer $promise(N, v)$
4. Si Proposer recibe $promise$ de la mayoría de Acceptors, crea la propuesta y envía a los Acceptors $propose(N, v)$
5. Si los Acceptors mantienen la promesa para la propuesta "N", aceptan el valor propuesto de " v " y envían $accept(N, v)$ tanto al Proposer como al Learner
6. El Learner comunica al cliente el valor consensuado y lo almacena según corresponda

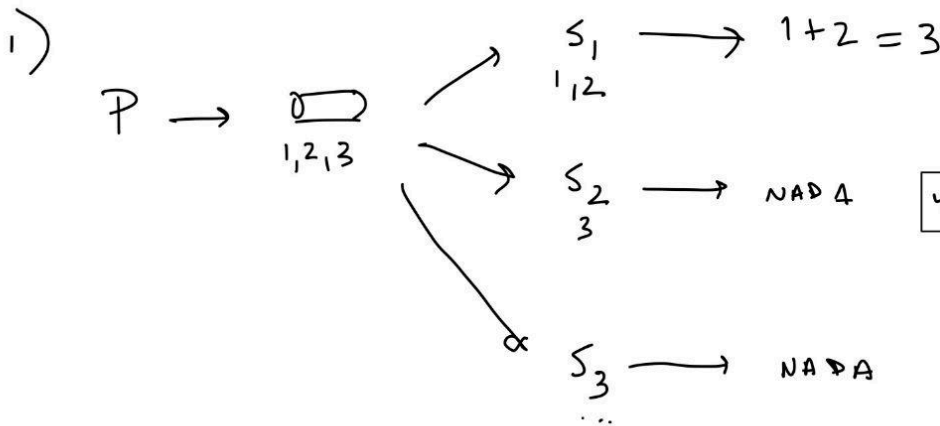
La fórmula de quórum es: $N \geq 2f + 1$; por lo cual, para que el algoritmo funcione...

- a) si se cae 1 proceso ($f = 1$), se necesitan $N \geq 3$ nodos
- b) si se cae 2 procesos ($f = 2$), se necesitan $N \geq 5$ nodos

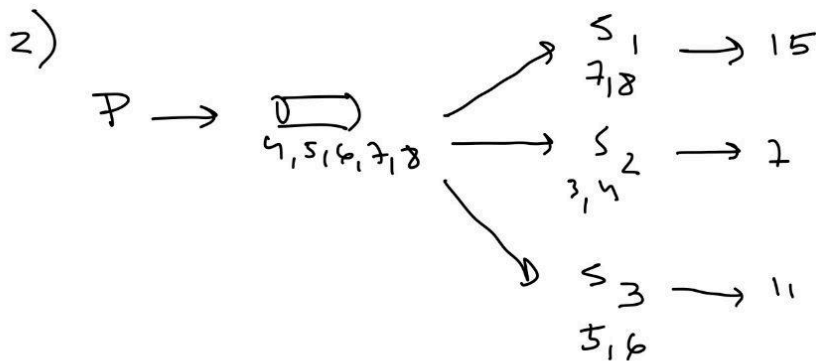
8. Utilizando una arquitectura Publisher-Subscriber, calcule la suma de todos los enteros incluidos en un vector de tamaño N, optimizando el uso de P procesos conocidos como disponibles. Utilice pseudocódigo o diagrama UML (secuencia, colaboración o actividades).



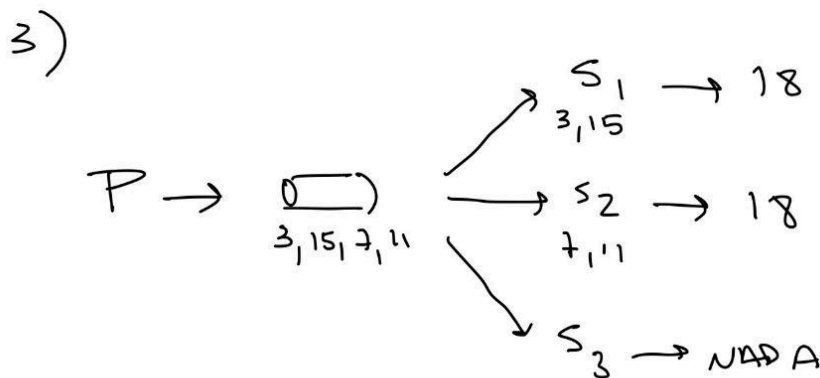
1	2	3	7	5	4	2	8
---	---	---	---	---	---	---	---



4	5	6	2	0	3		
---	---	---	---	---	---	--	--



3	15	7	11				
---	----	---	----	--	--	--	--



18	18	--
----	----	----

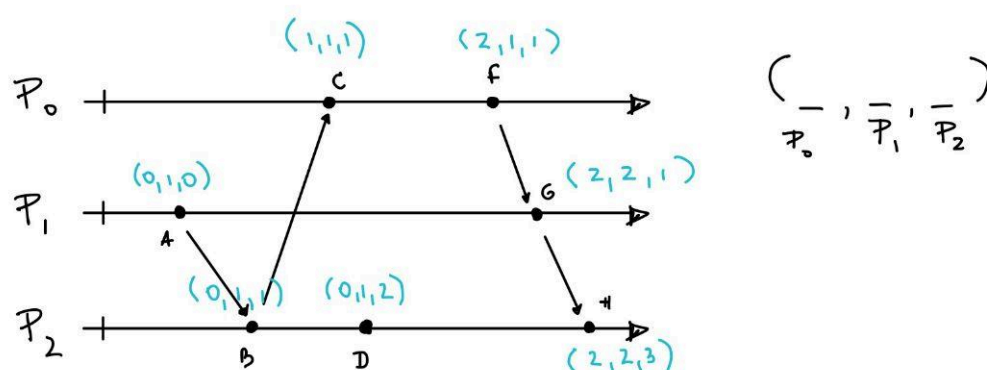
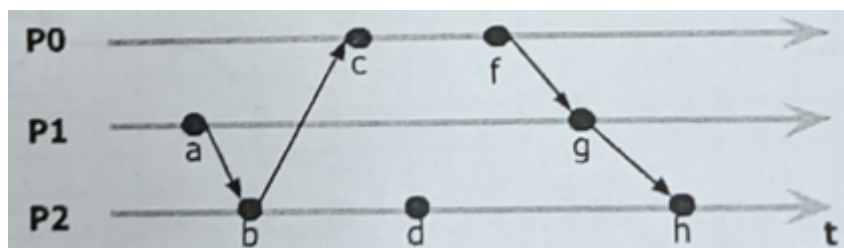
- 4) Luego se encolan dos "18", se suman de la misma forma y se agrega "36" al arreglo.
El publisher lee del arreglo un sólo número (el "36") y lo devuelve.

P
18, -

↳ sólo lee 1 elemento del arreglo
compartido → es la suma TOTAL

↳ devuelve **36**

9. Dado el siguiente diagrama de tiempo, calcule el valor de relojes lógicos para cada evento. Utilizando únicamente los vectores de relojes, establezca la relación: entre b y g, entre d y g. Justifique.



cada estado se calcula como:

local += 1

$$P_j = \max(e_j, msj_j)$$

$$B \rightarrow G \Leftrightarrow C(B) < C(G)$$

$$\Leftrightarrow (0, 1, 1) < (2, 2, 1)$$

$$\Leftrightarrow B_x < G_x \wedge B_y < G_y \wedge B_z < G_z$$

$$\Leftrightarrow 0 < 2 \wedge 1 < 2 \wedge 1 \leq 1$$

Pueden ser "<=" pero al menos uno debe ser "<"

~~$$\therefore C(B) \not< C(G) \wedge C(G) \not< C(B) \Rightarrow \text{los eventos son CONCURRENTES}$$~~

$$C(B) < C(G) \Rightarrow B \rightarrow G$$

$$D \rightarrow G \Leftrightarrow C(D) < C(G)$$

$$\Leftrightarrow (0, 1, 2) < (2, 2, 1)$$

$$\Leftrightarrow 0 < 2 \wedge 1 < 2 \wedge 2 \not< 1$$

$$\therefore C(D) \not< C(G) \wedge C(G) \not< C(D) \Rightarrow \text{los eventos son CONCURRENTES}$$

10. Diseñe una arquitectura que asegure la escalabilidad para un sistema con los siguientes requisitos

- Recibir usuario, timestamp y cambios de canales realizados en Argentina y Brasil por usuarios que poseen Smart TV de la marca LG y aceptó los términos y condiciones que incluyen monitoreo de uso.
- Consultar la cantidad de usuarios actualmente sintonizando cierto canal.
- Consultar los 5 canales más visualizados por los usuarios (mayor cantidad de minutos) en cierto rango de días.
- Consultar cantidad de usuarios con más de 1 visualización por día.

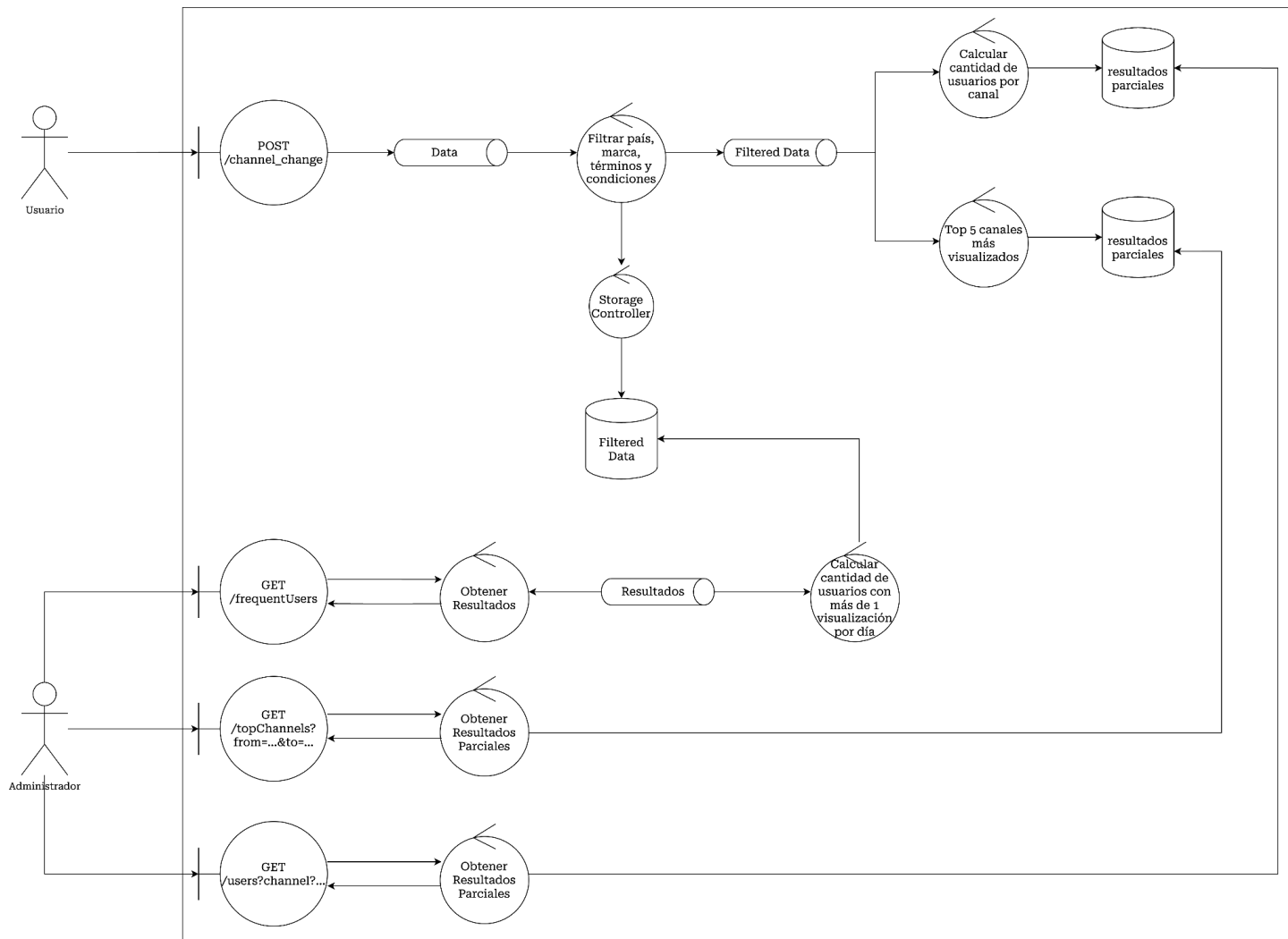
Resuelva el problema mediante un diagrama de vista física y su explicación.

```
POST / channel_change
  body : {
    userId : string (UUID)
    timestamp: datetime
    channel: int
    country: int
    TVbrand: string
    terms: bool
  }
```

```
GET / users ? channel = ...
  body : { total: int }
```

```
GET / topChannels ? from = ... & to = ...
  body : { topChannels : string [] }
```

```
GET / frequentUsers
  body : { total: int }
```



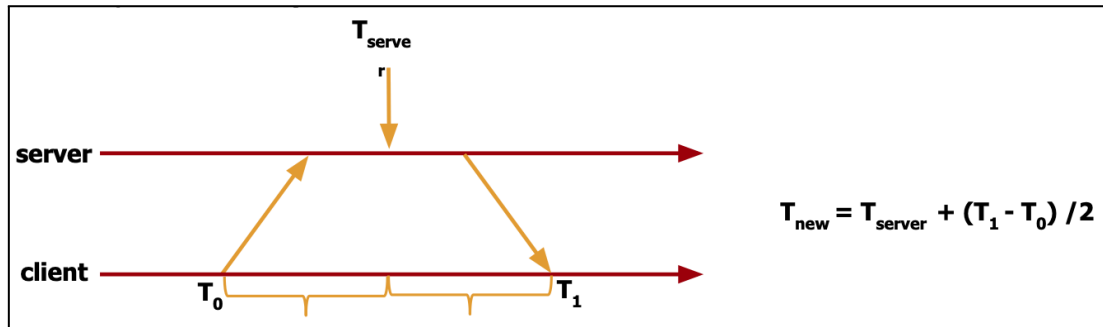
20220809

1. 🤖 Indique las ventajas y desventajas de utilizar comunicación por sockets TCP frente al pasaje de mensajes por colas mediante MOM en un sistema distribuido típico.

2. ¿A qué se refiere el concepto de *drift* de un reloj? Explique algún algoritmo para mitigar su efecto.

El drift de un reloj corresponde al desvío respecto de un reloj ideal o a un valor de referencia (conjunto de relojes) del sistema.

Para mitigar su efecto se puede utilizar el algoritmo de Cristian que calcula el valor del reloj actual como el valor del servidor al que hace el request más el delay cometido en la request efectivamente:



3. En qué se diferencia la exigencia de Orden Total vs Orden Causal para una serie de eventos distribuidos. Dé un ejemplo donde se respete el primero y a su vez se viole el segundo.

Orden Causal → todo mensaje que genere un nuevo mensaje, es visto de esta manera por todos los procesos respetando la causalidad:

si $\text{message}_1 \rightarrow \text{message}_2 \rightarrow \text{message}_3$

entonces $\text{delivery}_1 \rightarrow \text{delivery}_2 \rightarrow \text{delivery}_3$

Orden Total → todo conjunto de mensajes enviado a distintos receptores, son recibidos por ellos en el mismo orden:

si receptor_1 recibe message_2 y message_1

entonces receptor_2 recibe message_2 y message_1

Si $\text{delivery}_1 \rightarrow \text{delivery}_2$ pero se impone un orden total de forma que

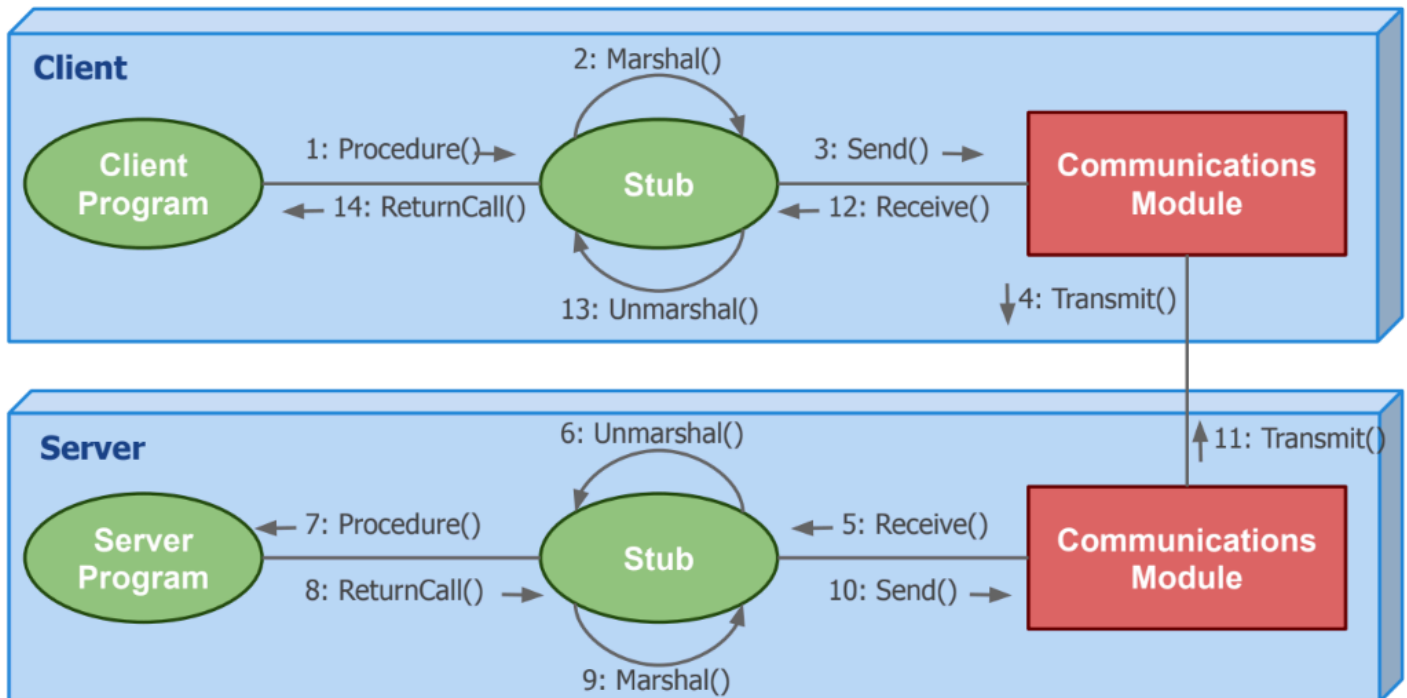
receptor_1 recibe message_2 y message_1

receptor_2 recibe message_2 y message_1

se viola el orden causal.

4. Explique cómo funciona el modelo RPC para arquitecturas distribuidas incluyendo el flujo de comunicaciones para un ejemplo propuesto.

Remote Procedure Call es un modelo por el cual el cliente ejecuta de forma remota en un servidor distintos procedimientos con la ilusión de que el procesamiento ocurre localmente.



5. ¿Qué componentes requiere una arquitectura para brindar elasticidad? Describa una situación donde este comportamiento sea deseable.

La elasticidad es la capacidad de un sistema de aumentar o disminuir sus recursos de manera dinámica en base a distintas métricas de demanda que varían a lo largo del tiempo.

Los componentes que son requeridos para brindar elasticidad a una arquitectura son:

- Load Balancer → balancear la carga (requests) entre los distintos nodos de un sistema distribuido
- Autoscaler → aumentar o disminuir los recursos físicos del sistema distribuido
- Automatic Monitoring → herramienta para monitorear el comportamiento de la carga durante un período de tiempo en particular

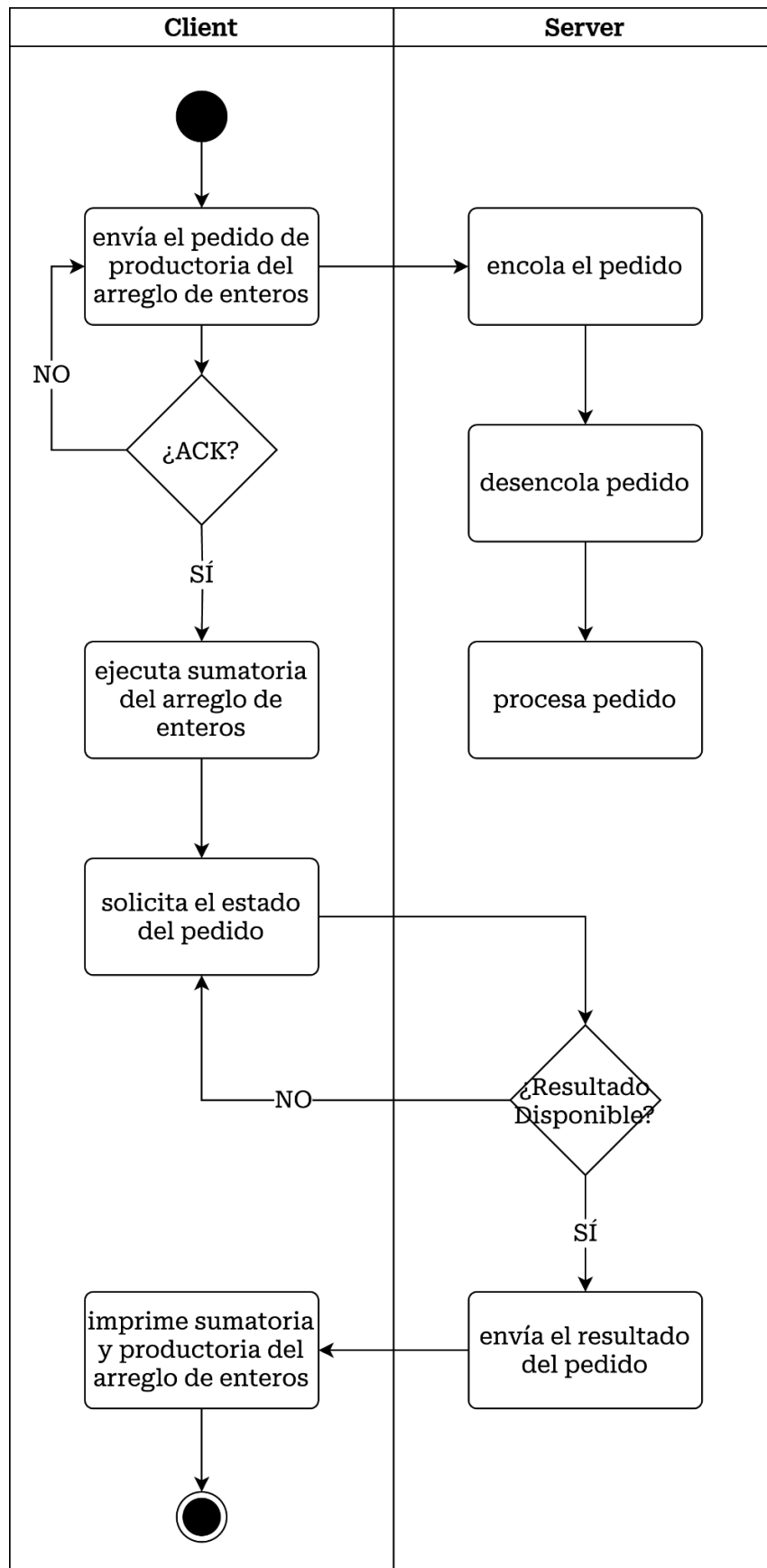
6. Considerando un escenario de particionamiento de datos en un sistema distribuido, explique al menos dos alternativas de enrutamiento de una consulta para poder ubicar la información deseada.

Hay 3 enfoques de enrutamiento de datos en un escenario donde la información de un sistema distribuido se encuentra particionada:

1. El cliente solicita la información a cualquier shard del sistema, si no la tiene le devuelve la ruta donde se encuentra efectivamente y el cliente procede a consultar en la ruta por la información.
2. El cliente solicita la información a un Router del sistema distribuido que le responde cuál es la ruta de la información efectivamente y el cliente procede a consultar en la ruta por la información.
3. El cliente conoce de antemano, por una función de partición, donde se encuentra la información y realiza la consulta al shard correspondiente.

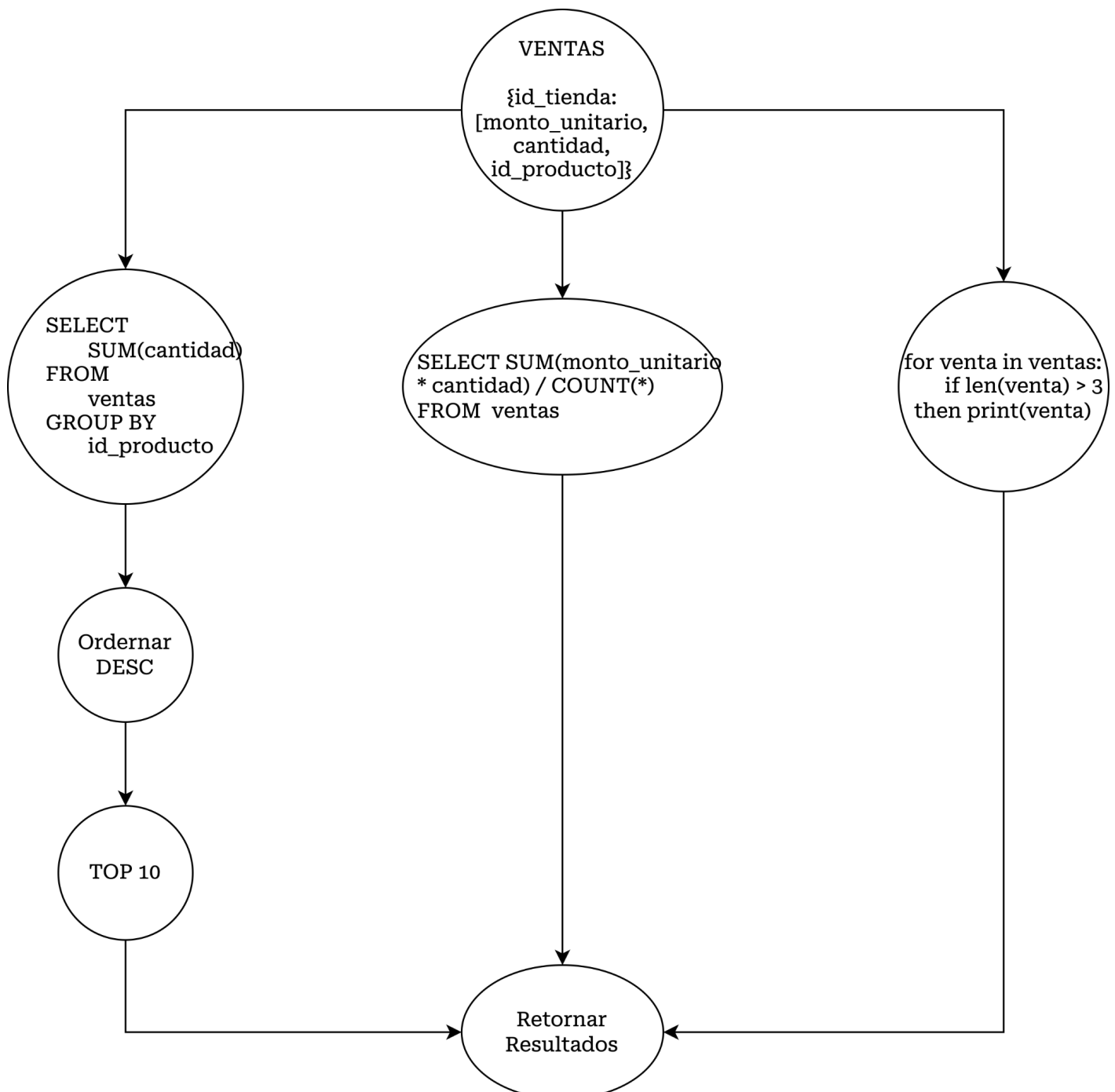
7.  Explique un algoritmo de exclusión mutua distribuida indicando sus ventajas y desventajas.

8. Utilizando una arquitectura Request-Reply, ejecute de forma remota la productoria de un arreglo de enteros de manera asincrónica. Mientras la ejecución transcurre, calcule la sumatoria. Por último, imprima ambos resultados. Utilice pseudocódigo o diagrama UML (secuencia, colaboración o actividades).



9. Ejemplifique el uso de Direct Acyclic Graphs (DAGs) al diseñar el procesamiento de datos de todas las ventas diarias de una cadena de tiendas de ropa. Cada venta posee ID de tienda y un arreglo con: monto unitario, cantidad y código de producto. Se pretende obtener los 10 códigos de producto más vendidos, el monto promedio de venta y la cantidad de ventas con más de 3 productos.

- venta = {id_tienda: [monto_unitario, cantidad, id_producto]}
- obtener
 - Q1 → 10 códigos de producto más vendidos
 - Q2 → monto promedio de venta
 - Q3 → cantidad de ventas con más de 3 productos



10. Diseñe una arquitectura que asegure la escalabilidad para un sistema con los siguientes requisitos

- a. Recolectar la información de ventas de pasajes de trenes de larga distancia de toda Argentina, desde las distintas estaciones que funcionan como puntos de venta para los servicios que prestan.
- b. Recibir listado actualizado de servicios a prestar indicando origen, destino, fecha y Cant. de asientos.
- c. Recibir pedido de compra con destino, fecha de partida, nombre de cada pasajero y foto de cada DNI.
- d. Permitir consultar fotos de DNI de pasajeros incluyendo origen y destino si se posee la fecha de partida.

Resuelva el problema mediante un diagrama de vista física y su explicación.

20230725

1. 🧐 Explique la necesidad del uso de protocolos binarios y no-binarios para la comunicación en un sistema distribuido. De un ejemplo de protocolos conocidos en estas dos categorías.

2. ¿Qué es un grupo de comunicación? ¿Qué utilidad presenta este concepto a la hora de diseñar un sistema escalable?

Un grupo de comunicación es un conjunto de clientes/procesos/entidades comunicados entre sí a través de distintos métodos (broadcast, multicast, unicast, anycast).

El concepto de grupo es útil a la hora de diseñar un sistema escalable debido a que su **creación y/o eliminación es sencilla** (los grupos son dinámicos). Además, un proceso puede **suscribirse o darse de baja** en un grupo según la intención que tenga de recibir o no un mensaje.

También es útil el **concepto de los distintos tipos de mensajes** que pueden ser enviados en un grupo:

- Unicast → comunicación 1 a 1
- Anycast → el mensaje llega a algún miembro del grupo
- Multicast → un mensaje llega a todos los miembros del grupo
- Broadcast → un mensaje llega a todos los miembros del sistema

Además hay **distintas topologías** que pueden representar la arquitectura de un grupo que son útiles al escalar un sistema (anillo, punto a punto, grupos jerárquicos, difusión centralizada, difusión descentralizada).

3.  Defina los conceptos de Estado Local y Estado Global de un sistema distribuido. Explique un algoritmo para realizar cortes consistentes de estado global.

4. Explique HDFS (Hadoop Distributed File System) y desarrolle los conceptos clave utilizados como factores de diseño.

HDFS es un sistema de archivos distribuido que permite **compartir elementos entre distintos usuarios** dando la **ilusión** de que dichos elementos se encuentran **disponibles localmente** para los usuarios. Está diseñado para funcionar en **hardware de bajo costo** y se considera un almacenador de datos en lugar de un File System ya que no soporta POSIX.

Los factores de diseño son:

- Prioriza **adaptarse** a los posibles **fallos de HW** antes que prevenirlos
- Favorece operaciones de **streaming** y datos volumétricos
- Favorece operaciones de **lectura** sobre escritura: write once, read many

5.  Explique un algoritmo de exclusión mutua distribuida:
Indique ventajas y desventajas del mismo en un caso concreto.

6. ¿Qué propone el teorema CAP? Según el teorema, ¿Es posible garantizar la tolerancia a fallos de un sistema distribuido?

El teorema enuncia que en un sistema distribuido sólo se pueden garantizar 2 de las siguientes propiedades al mismo tiempo:

- Consistencia (Consistency) → la información de todas las réplicas/nodos del sistema distribuido es equivalente
- Disponibilidad (Availability) → el sistema provee de una respuesta en todo momento (al menos hay una réplica/nodo funcionando)
- Tolerancia a las Particiones (Partition Tolerance) → el sistema sigue funcionando a pesar de que pueda fallar la comunicación entre réplicas/nodos

→ Consistencia + Disponibilidad: todos los nodos del sistema distribuido ven la misma información y están siempre disponibles, pero los nodos no siempre pueden comunicarse (se rompe el contrato de sistema distribuido)

→ Consistencia + Tolerancia a las Particiones: todos los nodos del sistema distribuido ven la misma información y siguen funcionando a pesar de que la comunicación entre ellos se pierda, pero la respuesta al usuario/cliente podría sufrir algún tardío ya que no siempre podría estar disponible el sistema distribuido

→ Disponibilidad + Tolerancia a las Particiones: el sistema siempre responde a los distintos pedidos del cliente y no deja de funcionar a pesar de que la comunicación entre los distintos nodos se pierda, pero puede ocurrir el caso de que los nodos vean información distinta

Se puede garantizar la tolerancia a fallos en un sistema distribuido mediante distintas estrategias de manejo, resiliencia, recuperación, redundancia y replicación.

Estrategias de Manejo:

- Fault Removal
- Fault Forecasting
- Fault Prevention / Avoidance

Resiliencia: mantener el sistema aceptable en presencia de fallos → GRACEFUL DEGRADATION (comportamiento aceptable que difiere del normal en presencia de fallos -"está mal, pero no está tan mal"-)

Recuperación: llevar al sistema a un estado correcto luego del fallo.

Redundancia: puede ser física, de información o del tiempo (lógica de reintentos)

Replicación o single point of failure, puede ser pasiva, activa o semi activa.

7. Explique los términos falla (fault), error y avería (failure) y cómo estos se relacionan con el concepto de tolerancia a fallos.

fault (falla) → condición no cumplida

error → estado incorrecto en el sistema

avería (failure) → comportamiento incorrecto de cara al usuario

Un rayo cósmico cae sobre un equipo y causa una **FALLA** en el sistema, cambiando un bit de la memoria RAM. El cambio de este bit provoca un **ERROR** pues hace que una variable del heap apunte a una dirección inválida en memoria. El sistema se **AVERÍA**, muestra un cartel de bloqueo al usuario y deja de funcionar.

En un sistema tolerante a fallos se busca que aún en la presencia de fallas, el sistema siga funcionando de cara al usuario sin llegar a convertirse en una avería.

8. Defina con pseudocódigo los pasos para extraer 31000 de una cuenta bancaria de número CA-1234 y acreditarlos en otra de número CA-S673 asumiendo la existencia de una plataforma de **Distributed Objects**. Realice un gráfico que ejemplifique la arquitectura de Distributed Objects y estado de la operación luego de ejecutarla.

```

-go distributedBankService.go

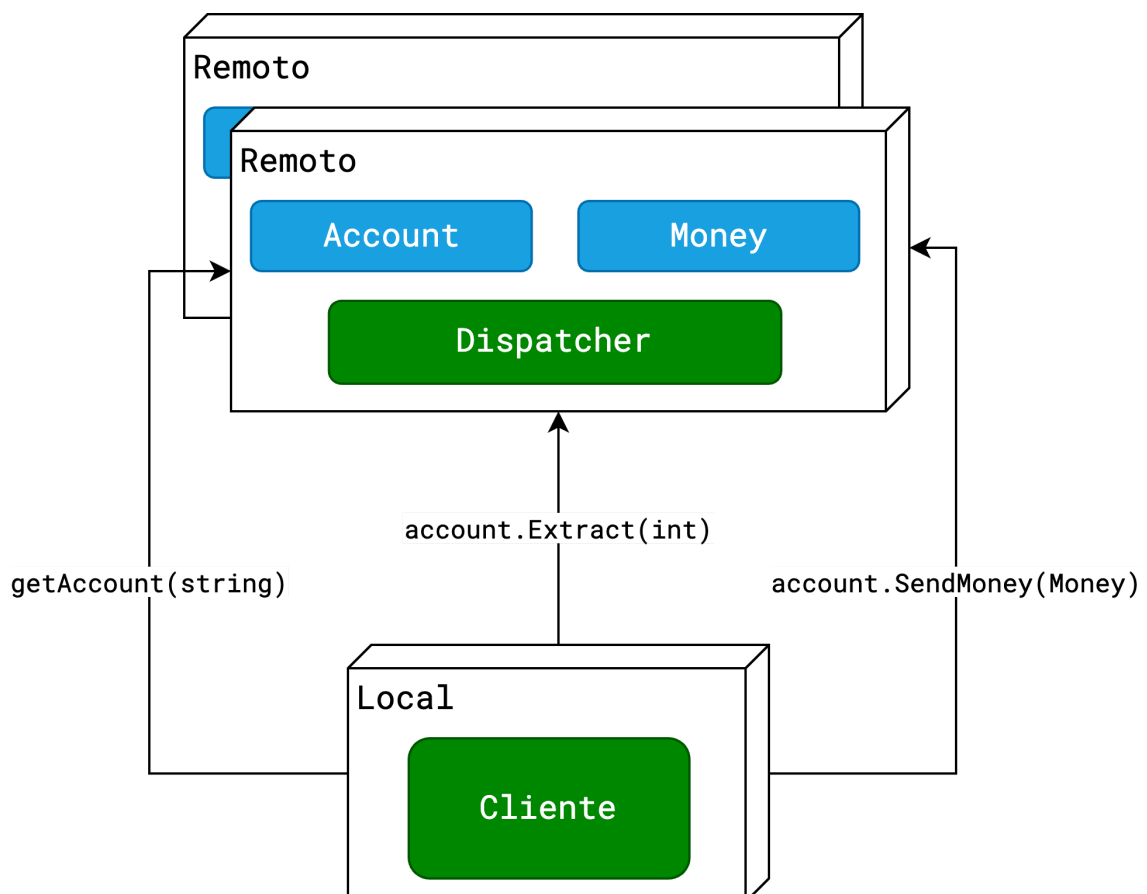
srv = getBankService()
account1 = srv.getAccount(CA-1234)

extracted, err = account1.Extract(31000)
if (err != nil) {
    throw("No hay suficiente dinero en la cuenta de origen")
}

account2 = srv.getAccount(CA-S673)

success = account2.SendMoney(extracted)
if (!success) {
    account1.SendMoney(extracted)
    throw("No se pudo realizar el depósito en la cuenta de destino")
}

```



9. Utilizando map-reduce para distribuir la carga, procese los eventos de log de una aplicación Web mediante un listado de tuplas (email, url_navegada, fecha) y calcule:

- a. Cantidad de navegaciones detectadas por día de semana (lu, ma,..., do)
- b. Listado de emails con más de 10 navegaciones registradas.

Utilice pseudo-código e indique supuestos de ser necesario.

a.

```
def map(list values):  
  
    // day → MON, TUE, WED, THU, FRI, SAT, SUN  
  
    // value → (email, url_navegada, fecha)  
  
    for v in values:  
  
        day = getDay(v.fecha)  
  
        emitIntermediate(day, 1)  
  
def reduce(string key, list values)  
  
    // key → MON || TUE || WED || THU || FRI || SAT || SUN  
  
    // values → lista de 1s asociados a la key  
  
    emit(key, len(values))
```

b.

```
def map(list values):  
  
    for v in values:  
  
        emitIntermediate(v.email, 1)  
  
def reduce(string key, list values)  
  
    // key → xxx@xxx.com  
  
    if len(values) >= 10  
  
        emit(key, len(values))
```


10. 🤖 Diseñe una arquitectura que asegure la escalabilidad para un sistema con los siguientes requisitos:

- a. Recibir mensajes WhatsApp incluyendo campos con tipo (texto o audio), contenido (string o binario), teléfono origen y nombre de usuario para procesar las consultas del canal oficial de Anses.
- b. Procesar todos los mensajes de audio para obtener el texto a partir del audio recibido.
- c. Identificar palabras clave y usar tablas p/calc. área, siendo
 - i. Jubilaciones y Pensio.
 - ii. Asignaciones Fam.
 - iii. Otras
- d. Crear un ticket con el contenido mediante POST en sistema externo y retornar ID al usuario.
- e. Permitir la consulta de tickets por números con cierta palabra clave y por área en cierto rango de fechas.

Resuelva el problema mediante un diagrama de vista física y su explicación.

20240708

1. ¿En qué consiste el modelo de Work-Span para análisis de paralelización de tareas y camino crítico?

El modelo de work-span, a diferencia del de Amdahl parte de la premisa de que no todas las tareas paralelizables pueden ser ejecutadas al mismo tiempo (**paralelismo imperfecto**), por lo tanto es un modelo más cercano a la realidad.

Propone una cota superior e inferior para el cálculo del Speed Up.

Utiliza dos métricas principales:

- $T_1 \rightarrow$ representa el tiempo total que llevaría ejecutar todas las tareas en un sólo procesador
- $T_\infty \rightarrow$ representa el tiempo total que llevaría ejecutar todas las tareas en una cantidad infinita de procesadores; este tiempo está dado por la longitud del camino crítico del mapa de tareas (secuencia de ejecución de tareas dependiente entre sí más larga)

2. ¿Qué se entiende por “Atomicidad de Mensajes” en el contexto de grupos de comunicación? ¿Con qué estrategias es posible garantizarla?

La atomicidad de mensajes refiere a que un mensaje tiene la garantía de ser entregado a todos los miembros de un grupo, o a ninguno en caso de existir un error en la comunicación.

Para garantizar que un mensaje es recibido por un miembro del grupo se utilizan los “ACK” de mensajes.

Para garantizar la atomicidad de mensajes, se utilizan:

- recepción de ACKs
- lógicas de reintento ante
 - caída de receptores
 - caída de emisor
 - no recepción de ACK
 - no recepción de mensajes

3. Defina 2 protocolos distintos para la transmisión de paquetes considerando un registro que posee un int y un arreglo de flotantes. Compare ventajas y desventajas de cada protocolo.

Protocolo 1: BINARIO → 4 posiciones para el int del largo del mensaje, seguido del mensaje (int + arreglo de floats)

VENTAJAS

- + mensajes ligeros
- + velocidad de transmisión es más rápida
- + no hay un empaquetamiento “extra” (el mensaje viaja por los canales de comunicación en binario)

DESVENTAJAS

- su lectura requiere deserialización → es difícil leerlo a simple vista
- la serialización / deserialización requiere un estándar de arquitecturas (se debe decidir sobre big endian o little endian dependiendo de la comunicación del sistema)

Protocolo 2: TEXTO → mensaje en formato json

VENTAJAS

- + su lectura no requiere deserialización → es fácil leerlo a simple vista
- + el formato ya se encuentra definido y es estándar (json)

DESVENTAJAS

- mensajes muy pesados
- velocidad de transmisión puede ser considerablemente lenta
- el mensaje debe ser empaquetado a formato binario para ser transmitido por los canales de comunicación entre sockets

4.  Explique HDFS (Hadoop Distributed File System) y desarrolle los conceptos clave utilizados como factores de diseño.
5.  Describa un algoritmo de exclusión mutua distribuida que no utilice un servidor central.
6.  Explique cómo se relacionan los conceptos de fallo (fault), error y avería (failure). Indique luego la importancia de esta relación para el estudio de sistemas tolerantes a fallos.
7.  Explique un algoritmo de consenso para que 4 procesos puedan llegar a un acuerdo sobre cierta variable que ellos definen. ¿Cómo actúa el algoritmo al caerse 1 proceso? ¿Y al caerse 2 procesos?

8.   Detalle de un algoritmo de consenso que permita que 3 procesos concuerden en el valor de altitud de un dron en pleno vuelo. Muestre paso a paso cómo funciona el algoritmo considerando que los procesos A,B miden 10m mientras que C mide 9m.

USAR GENERALES BIZANTINOS

9. 🧙 Dada una librería basada en request-reply, realice un pequeño sistema que realice la tarea de calcular el hash de un archivo binario leyendo sus bloques de 4 bytes y aplicando sucesivamente la operación XOR a medida que se leen más bloques. Distribuya la carga de cómputo entre N servidores conocidos. Utilice pseudocódigo o diagrama UML (secuencia, colaboración o actividades) para explicar la solución e indique supuestos de ser necesario.
10. 🧙 Diseñe una arquitectura que asegure la escalabilidad para un sistema con los siguientes requisitos
- Permitir la asignación de Aires Acondicionados inteligentes en LATAM para la captura de datos: fecha última medición, minutos en uso desde última medición, temprana medida, ubicación aproximada (lat, long).
 - Asegurar la captura de datos con una frecuencia de 10 minutos.
 - Permitir la consulta de un reporte con ciudades conocidas y temperatura min, max, avg, medidas en las últimas 24hs.
 - Permitir la generación de una alarma y reporte por email al dueño del dispositivo si se detecta el uso excesivo: más de 12 horas de uso durante las últimas 24 horas.

Resuelva el problema mediante análisis de volumen, endpoints y vista física con su explicación.

20240716

1. 🧙 Suponga que debe implementar un protocolo de health-check de server a clientes mediante sockets. ¿Qué condiciones se deberían cumplir para que sea más conveniente utilizar UDP frente a TCP?
 - Lógica de reintentos cada cierto tiempo en caso de no recibir una respuesta del cliente para asegurar que todos los mensajes llegan efectivamente
 - Esperar una respuesta del cliente indicando que está vivo efectivamente
 - No es necesario mantener una conexión abierta estable entre cliente - servidor

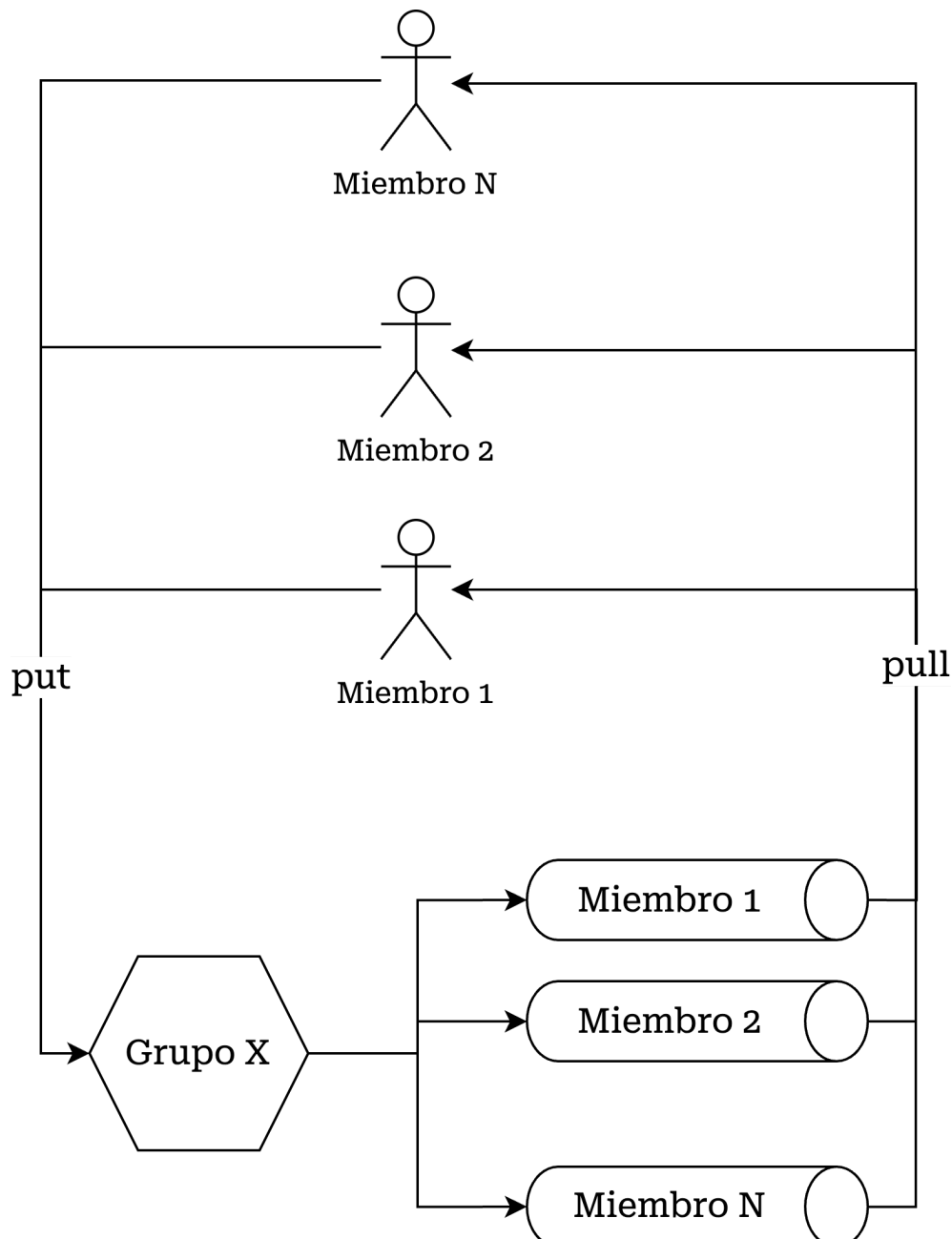
2.  Explique la utilidad de los algoritmos de relojes lógicos de Lamport y vectores de relojes (lógicos) explicados en clase. ¿Para qué sirve cada uno?

3. Describa cómo implementaría un middleware que garantice la comunicación de grupo con las siguientes condiciones:

1. sólo existen tres grupos: x, y, z
2. los miembros de los grupos se pueden unir en cualquier momento para recibir mensajes
3. sólo se permite enviar mensajes a todos los miembros de tu grupo

Implementaría un Message Oriented Middleware Centralizado.

En el servidor existirían 3 exchanges de tipo fan out (uno por cada grupo), y 1 a N colas para cada miembro del grupo. Los miembros que pertenezcan a un grupo se suscribirán a la cola correspondiente. Los miembros que quieran enviar un mensaje a su grupo harán un “put” del mensaje al exchange quien se encargará de propagar dicho mensaje a todas las colas que estén suscriptos a él.



4. 🤖 Explique el beneficio de la replicación de datos y desarrolle una estrategia de implementación.

- Respaldo/Backup de información en caso de pérdida de la información original
- Al tener la misma información en varios lugares distintos, se pueden mover los centros de almacenamiento acercando el contenido en caso de corresponder (ejemplo: CDNs) con el fin de disminuir la latencia de acceso a los datos **Principio de Proximidad**
- Se pueden distribuir las requests entre los distintos centros de almacenamiento de datos para no saturar a un único equipo

Una estrategia de implementación podría ser Leader Based: una réplica máster se encarga de mantener actualizadas las réplicas que funcionan como espejo a ella. La lectura se puede realizar en cualquiera de los nodos (tanto el máster como las réplicas). La escritura sólo es realizada por el nodo líder.

5.  Explique un algoritmo de Distributed Shared Memory (DSM) donde se utilice replicación de páginas. Indique un caso de uso donde el algoritmo elegido sería ventajoso y otro donde sería perjudicial.

6. 🤖 ¿En qué consiste la propiedad de Elasticidad de un sistema distribuido? ¿Qué elementos mínimos son necesarios para garantizarla?

7. El estudio de sistemas confiables (*dependable*) implica la revisión de varias propiedades que impactan en la tolerancia a fallos del sistema. Detalle los conceptos clave de al menos una de ellas.

Para que un sistema distribuido sea confiable debe garantizar las siguientes propiedades:

- Disponibilidad == Availability → capacidad del sistema para reparar/enmascarar fallas de manera tal que el sistema esté no disponible la menor cantidad de tiempo posible (esto se define en el SLA)
- Seguridad == Safety → capacidad del sistema de prevenir acciones maliciosas/accidentales
- Mantenibilidad == Maintainability → capacidad del sistema para ser mantenido por los administradores
- Confiabilidad == Reliability → confianza sobre el sistema ante eventuales fallas

Disponibilidad → ML sólo se cae 1' por año.

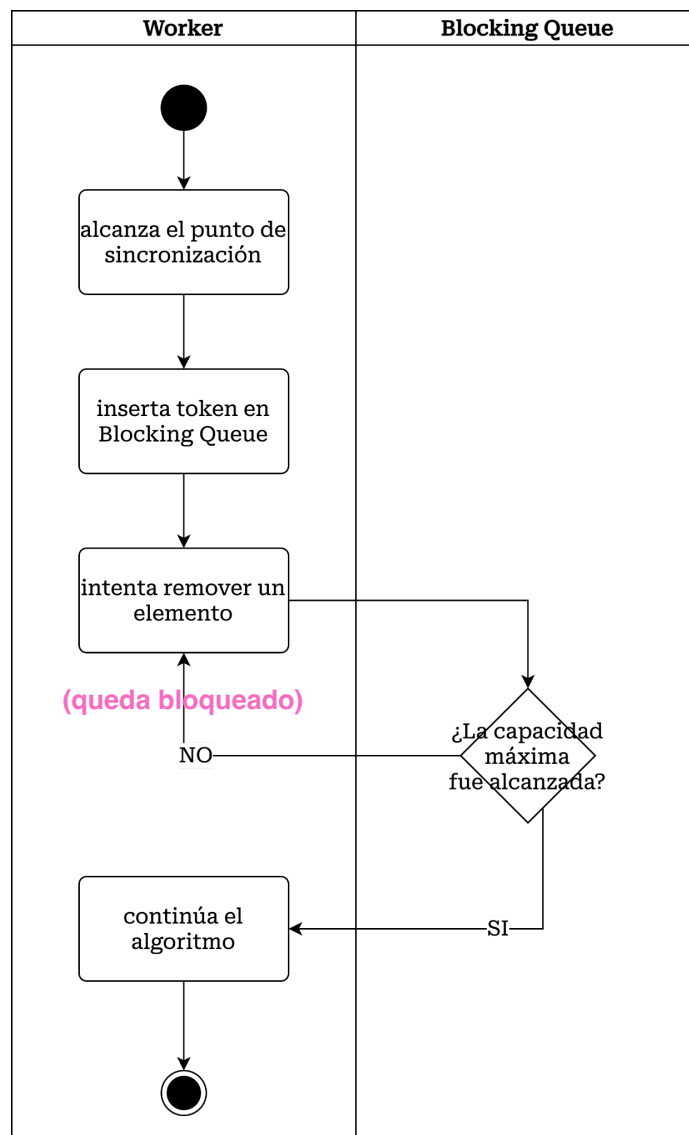
Seguridad → MP tiene autenticación 2fa además del resto de medidas de seguridad que toma al incorporar cuentas bancarias, realizar compras, etc.

Mantenibilidad → suponemos que ML/MP es fácil de mantener por ellos mismos. En lo que respecta al usuario no hay grandes trabas para publicar, comprar, vender, etc.

Confiabilidad → se que cuando ML/MP se cae todo va a seguir funcionando cuando se levante nuevamente.

8. 🤖 Utilizando blocking-queues, implemente una barrera para que 3 procesos llamados workers puedan sincronizarse en cierto punto del algoritmo que ejecutan. Detalle supuestos de ser necesario. Utilice pseudocódigo o diagrama UML (secuencia, colaboración o actividades).

1. Se crea una blocking queue con capacidad igual al número total de procesos, (3 en este caso).
2. Cuando el proceso llega al punto de sincronización, inserta un token en la cola (put()), luego procede a remover un elemento (take()). Si la capacidad máxima de la blocking queue no fue alcanzada (es decir, hay procesos que aún no pushearon un token), el proceso quedará bloqueado al hacer take().
3. Cuando el último proceso llega al punto de sincronización inserta el último token y la capacidad máxima de la queue es alcanzada. Esto provoca el desbloqueo de los procesos al intentar remover un elemento de la blocking queue y el algoritmo avanza.



9. 🤖 Defina con pseudocódigo los pasos para consultar el estado de una suscripción de Netflix con número NF-1234, obtener todos los serial numbers de dispositivos permitidos en la suscripción y datos de alta en otra suscripción número NF-12345 asumiendo la existencia de una plataforma de Distributed Objects. Realice un gráfico que ejemplifique dicha arquitectura y el estado de los objetos.
10. 😊 Diseñe una arquitectura que asegura la escalabilidad para un sistema con los siguientes requisitos:
1. Permitir el reporte ciudadano de infracciones de tránsito presenciadas en vía pública en CABA.
 2. El reporte consta de una foto e indica el tipo de infracción presenciada (“vehículo mal estacionado”, “cruce en rojo”, “choque con fuga”) y una descripción de lo ocurrido.
 3. Reconocer el vehículo del reporte y los datos de su titular, dejando registro de los mismos.
 4. Analizar automáticamente la foto para el tipo “cruce en rojo” y emitir una multa por email al titular en caso de validar la infracción.
 5. Permitir consulta para evaluación manual de los tipos restantes y para auditoría en general por 10 años.
 6. Notificar por email al ciudadano sobre el estado de su reporte (hubo/no hubo infracción) a los 7 días.

Resuelva el problema mediante análisis de volumen, endpoints y vista física con su explicación.

20241219

1.  Suponga que debe implementar un protocolo de health-check de server a clientes mediante sockets, ¿qué condiciones se deberían cumplir para que sea más conveniente utilizar TCP frente a UDP?

2. Explique la utilidad de los algoritmos de relojes lógicos de Lamport y de vectores de relojes (lógicos) explicados en clase. ¿Para qué sirve cada uno?

Tanto los relojes lógicos de Lamport como los vectores de relojes lógicos son utilizados para ordenar los eventos de los distintos procesos en un sistema distribuido y establecer una causalidad entre ellos.

La diferencia principal entre ambos enfoques es que el algoritmo de Lamport utiliza un escalar por proceso, mientras que el vector de relojes hace uso de un vector de escalares donde cada posición del vector corresponde al estado actual de cada uno de los procesos del sistema distribuido. En este sentido, el algoritmo de **Lamport** captura muy poca información y **sólo es útil cuando se necesita un orden parcial de los eventos y conocer si son paralelos efectivamente o no**. Ahora bien, el **vector de relojes** proporciona más información de cada uno de los procesos y **permite identificar eventos concurrentes**.

3.  ¿Qué se entiende por “Atomicidad de Mensajes” en el contexto de grupos de comunicación? ¿Con qué estrategias es posible garantizarla?

4. Asumiendo la existencia de un equipo de hasta 10 agentes de atención al cliente en una empresa telefónica, defina cómo implementar un middleware distribuido que permita a los agentes llamar al siguiente número en fila. ¿Dónde almacenaría la información relacionada al último número llamado?

Middleware recibe requests via sockets. Internamente tiene un load balancer que balancea los distintos pedidos entre las 10 colas asociadas a cada agente del sistema. La idea es que ninguna cola se encuentre saturada de pedidos y el trabajo se distribuya de forma uniforme.

El último número de llamado podría guardarse en el middleware para llevar un registro de la cantidad de requests recibidas/procesadas.

5.  Describa dos estrategias útiles para garantizar tolerancia a fallos en una arquitectura distribuida. Dé un ejemplo de arquitectura donde dichas estrategias deben ser utilizadas.
6.  El estudio de sistemas confiables (*dependable*) implica la revisión de varias propiedades que impactan en la tolerancia a fallos del sistema. Detalle los conceptos clave de al menos una de ellas.
7.  Explique una arquitectura de Distributed File System (DFS) que conozca. Indique un caso de uso donde elegir dicho mecanismo sería ventajoso y otro donde sería perjudicial.

8. Utilizando request-reply, implemente una barrera para que 3 procesos llamados workers puedan sincronizarse en cierto punto del algoritmo que ejecutan. Detalle supuestos de ser necesario. Utilice pseudocódigo o diagrama UML (secuencia, colaboración o actividades).
- request reply sincrónico (que los procesos esperen por la respuesta)
 - el server devuelve ACK únicamente cuando tiene la misma cantidad de requests encoladas como de procesos en el sistema (3)

9. 🤖 Utilizando map-reduce, procese los eventos de mensajes recibidos en un chatbot mediante un listado de tuplas (messageId, clientPhoneNumber, text, date, time) y calcule
- cantidad de clientes atendidos por día del mes (1, 2, ..., 30 ó 31)
 - listado de teléfonos con más de 10 mensajes recibidos en un mismo día, utilice pseudocódigo
10. 🤖 Diseñe una arquitectura que asegure la escalabilidad para un sistema con los siguientes requisitos:
- Tomar los pedidos de café y *cookies* en terminales AUTOSERVICIO de una cadena multinacional con sucursales en las capitales de todos los países, para que empleados preparen y cobren pedidos en caja
 - Emitir una comanda de pedido con cantidad y producto en las terminales COCINA y un ticket de cobro con precio unitario y total en las terminales CAJA de la sucursal en cuestión
 - Recibir de forma centralizada la lista de productos y fotos a vender en las terminales en cada sucursal
 - Recibir de forma centralizada los pedidos de cambio de precio unitario para cada sucursal
 - Permitir consultar ventas totales por día y sucursal

Resuelva el problema mediante análisis de volumen, endpoints y vista física con su explicación.

20250213

1. ¿Qué es el *throttling* o descarte de paquetes? ¿Tiene efectos negativos?

Ante una sobrecarga del sistema ó de requests el sistema recurre a disminuir la cantidad de requests procesadas o mensajes entregados o bien descartando paquetes con el objeto de liberar o bien disminuir el flujo de datos sobre el canal de comunicación.

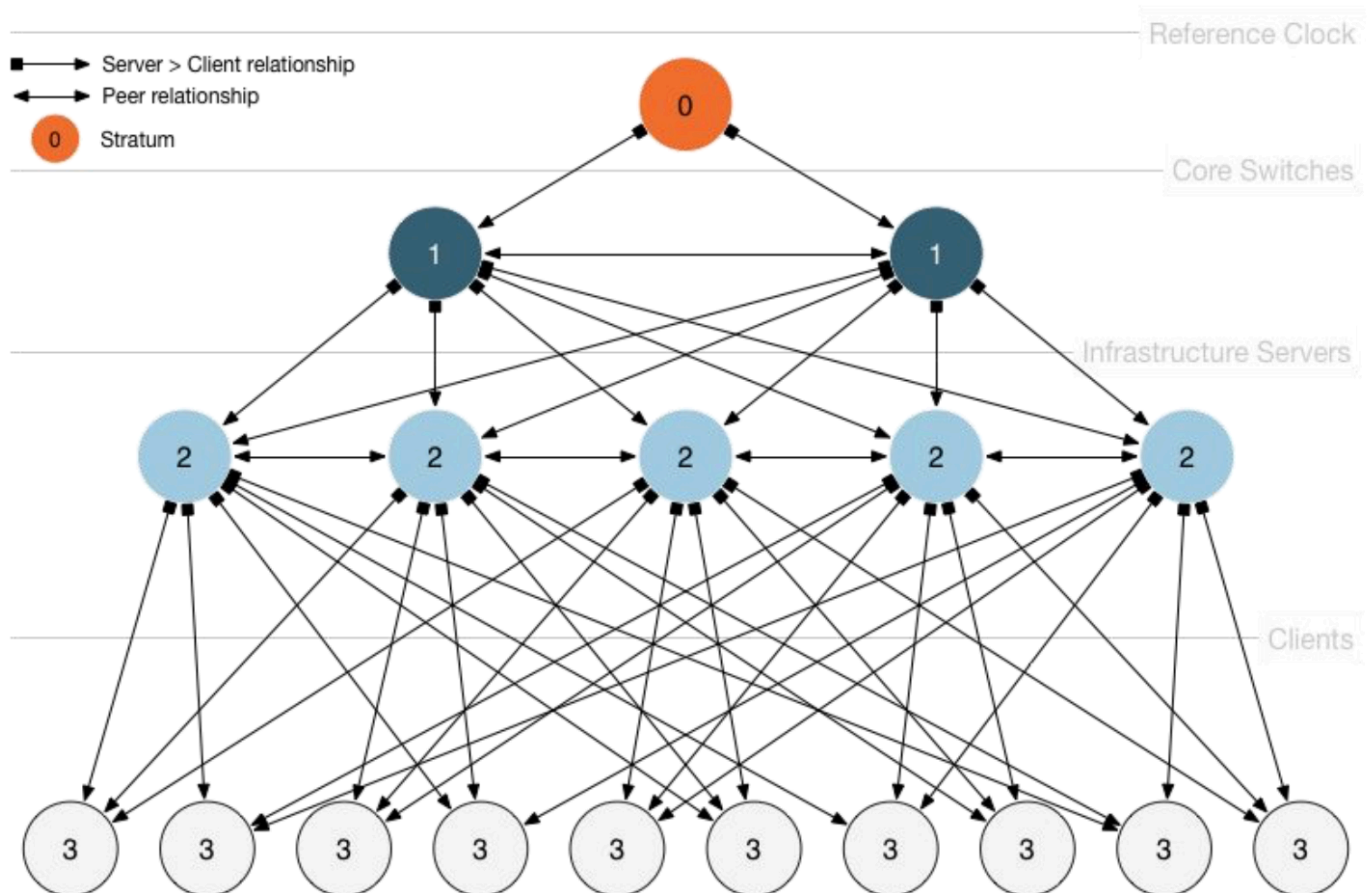
Los efectos negativos podrían ser que:

1. la información podría perderse (si se utiliza un protocolo UDP en crudo, por ejemplo)
2. los emisores tienen la responsabilidad de controlar no sólo que el mensaje sea recibido si no que sea enviado efectivamente y debería implementar una política de reintento en caso de una falta de respuesta

2. Describa los conceptos clave de Network Time Protocol (NTP) usado para sincronización de relojes en sistemas distribuidos. ¿Por qué se dice que este protocolo es escalable?

El protocolo se utiliza para sincronizar los relojes de los clientes aún en presencia de delays en la red. Existe un gran número de clientes sincronizados de forma frecuente. Tiene en cuenta los efectos de drift. Sobrevive a largas caídas de conectividad (alta disponibilidad).

Está basado en estratos para la sincronización de los relojes:



El estrato 0 es el master clock. Los estratos 1 y 2 propagan la información con el nivel de abajo y entre ellos. El estrato 3 es el equipo final que necesita la sincronización con el master clock, los nodos en este estrato no se comunican entre ellos si no que toman la información que les da el estrato anterior.

Esta jerarquía muestra por qué el protocolo es escalable ya que se pueden sumar fácilmente clientes/procesos/equipos a un estrato sincronizándose con los estratos superiores.

3.  Describa 3 ventajas del uso de middlewares en sistemas distribuidos.
4.  Describa cómo implementaría un middleware que cumpla con las siguientes características:
 - a. Solo existen 3 grupos de comunicación (x, y, z)
 - b. Los miembros se pueden sumar y dar de baja de un grupo en cualquier momento
 - c. Los mensajes pueden ser mandados a “todos” los miembros del grupo o a uno en particular

5. Describa el modelo de Pipeline-Filters para procesamiento de datos. Describa cómo funcionan los modelos de Worker por Filter y Worker por Dato.

En el modelo de pipeline filters los datos de entrada forman un flujo donde distintos filtros se conectan entre sí para procesar los datos de manera secuencial.

source -pipeline-> filter -pipeline-> filter -pipeline-> filter -pipeline-> sink

En el modelo de worker por filtro, cada filtro es un worker que procesa el dato que viene del pipeline y lo envía a la próxima etapa.

En el modelo de worker por dato, existe un único worker que acompaña al dato/ítem que lo acompaña a lo largo de todo el proceso, aplicando los filtros paso a paso.

6. Describa cómo se implementa la redundancia para la tolerancia a fallas. Dé un ejemplo en donde esta estrategia garantice tolerancia a fallos y un ejemplo en donde no se garantice.

La redundancia puede ser

- física → HW, disco rígido de backup
- de información → réplica de datos, backup de información
- de tiempo → reintentos

Para tolerar fallos con redundancia física, 2 nodos replicados con un load balancer que distribuya requests y ante la caída de uno de los nodos redirige las requests al otro disponible (replicación pasiva).

Otro ejemplo podría ser tener dos discos físicos con el objeto de que uno funcione como respaldo ante la posible ruptura del primero. Si el primero se rompe de forma inesperada y el segundo no estaba siendo espejado correctamente, perderíamos toda la información y la función del segundo disco que hacía de respaldo sería obsoleta.

7. Explique un modelo de Distributed Shared Memory (DSM) en donde se use replicación de páginas. Explique por qué este modelo sería mejor que el enfoque Naive de DSM.

El modelo de replicación de páginas es mejor que el enfoque Naive por la baja performance que tiene este último ya que sólo hay una partición de memoria -que se encuentra obviamente centralizada en el servidor-. Todos los clientes que quieren acceder a una porción de memoria, toman todo el recurso bloqueando a los demás que quieren acceder de igual forma. En la replicación de páginas, al estar segmentado el almacenamiento se permite que más de un cliente pueda acceder a más porciones de memoria ya que la paginación da la ilusión que se tienen *más bloques de memoria*.

8. 🤖 Pregunta de vectores lógicos (no tenemos foto) → no lo hacemos
9. 🤖 Detalle cómo sería una implementación de map-reduce que reciba las llamadas que ocurren por día en una compañía telefónica. Cada llamada cuenta con `nro_origen`, `nro_destino`, `timestamp`, `duracion` (en segundos). Resuelva las siguientes consultas:
 - a. Duración total de las llamadas para cada `nro_origen`
 - b. Determinar el `nro_destino` que recibió mas llamadas
 - c. Duración promedio de cada llamada
10. 🤖 Diseñe una arquitectura que asegure la escalabilidad para un sistema con los siguientes requisitos
 - a. Recolectar la información de ventas de pasajes de trenes de larga distancia de toda Argentina, desde las distintas estaciones que funcionan como puntos de venta para los servicios que prestan.
 - b. Recibir listado actualizado de servicios a prestar indicando origen, destino, fecha y Cant. de asientos.
 - c. Recibir pedido de compra con destino, fecha de partida, nombre de cada pasajero y foto de cada DNI.
 - d. Permitir consultar fotos de DNI de pasajeros incluyendo origen y destino si se posee la fecha de partida.

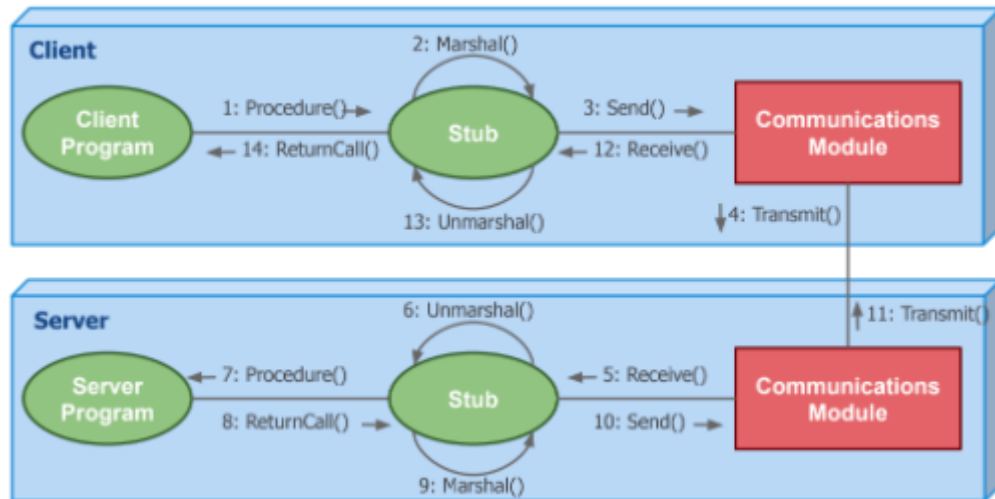
Resuelva el problema mediante un diagrama de vista física y su explicación.

20250220

1.  ¿En qué consiste el modelo de Work-Span para análisis de paralelización de tareas y camino crítico?

2. Explique cómo funciona el modelo RPC en un caso ficticio donde su sistema se conecta con Aerolíneas para
- obtener token para user/pass
 - consultar vuelos disponibles (número/precio) para cierto origen/destino/fecha

Considere que la consulta puede demorar minutos. Grafique el flujo elegido.



3. El contrato de una interfaz puede estar orientado a entidades o a procesos. Describa ambos modelos y dé un ejemplo de al menos dos endpoints de cada estilo para una API.

Orientado a Entidades:

- Expone los datos como recursos accesibles mediante operaciones CRUD (Create, Read, Update, Delete)
- Modelo más cercano a la estructura de la base de datos y representa directamente las entidades del dominio
- Ejemplos
 - GET /clientes/{id}
 - POST /pedidos

Orientado a Procesos:

- Expone acciones o flujos de negocio que reflejan cómo se utilizan los datos en un contexto real
- Ejemplos
 - POST /pedidos/{id}/confirmar
 - GET /clientes/{id}/compras?desde=2024-01-01&hasta=2024-12-31

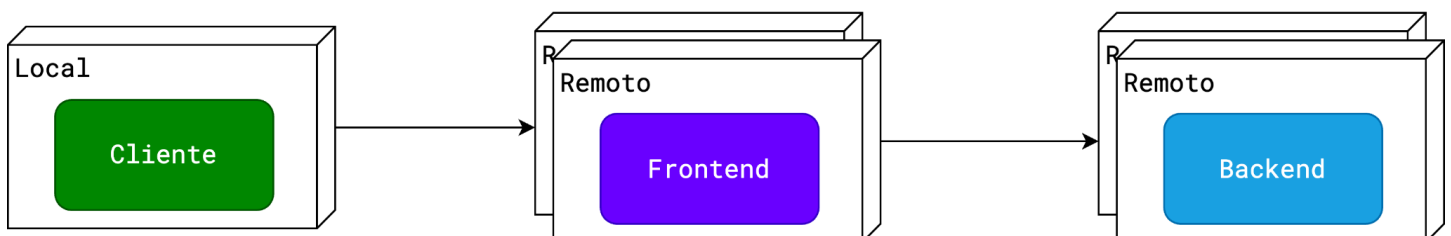
4. ¿Cómo se calculan los 9s de disponibilidad de un sistema con 2 de frontend idénticos, accedidos por los usuarios y que se comunican con 2 nodos de backend idénticos. Asuma $P(availability) = 0,95$ para cada nodo y que el balanceo de carga es transparente con load balancers que no se caen.

Primero calculamos la disponibilidad de un cluster, cualquiera sea:

$$\begin{aligned}
 P(availability) &= 1 - P(failure) \\
 &= 1 - P(failure\ 1 \ \& \ failure\ 2) \\
 &= 1 - (0.05 * 0.05) \\
 &= 1 - 0.0025 \\
 &= 0.9975 \rightarrow \text{disponibilidad de un cluster (de ambos FE y BE} \\
 &\quad \text{porque son idénticos y tienen la misma } P(available))
 \end{aligned}$$

$$\begin{aligned}
 P(system\ availability) &= P(frontend\ av.) * P(backend\ av.) \\
 &= 0.9975 * 0.9975 = 0.99500625 \approx 0.9950
 \end{aligned}$$

La disponibilidad del sistema es de 99.5 %.



5.  Describa dos estrategias útiles para garantizar tolerancia a fallos en una arquitectura distribuida. Dé un ejemplo de arquitectura donde dichas estrategias deben ser utilizadas.
6.  Explique la utilidad de los algoritmos de relojes lógicos de Lamport y vectores de relojes (lógicos) explicados en clase. ¿Para qué sirve cada uno?
7.  Explique HDFS (Hadoop Distributed File System) y desarrolle los conceptos clave utilizados como factores de diseño.





Utilizando blocking-queues, implemente un procesamiento con arquitectura productor-consumidor donde exista una barrera para que 3 procesos consumidores se esperen entre sí luego de haber procesado 2 ítems cada uno. Utilice pseudocódigo o diagrama UML (secuencia, colaboración o actividades).

9. ♻ Ejemplifique el uso de Direct Acyclic Graphs (DAGs) al diseñar el procesamiento de datos de todas las ventas diarias de una cadena de tiendas de ropa. Cada venta posee ID de tienda y un arreglo con: monto unitario, cantidad y código de producto. Se pretende obtener los 10 códigos de producto más vendidos, montos min-max-avg de ventas, y la cantidad de ventas que incluían más de 3 productos.

10. 🤖 Diseñe una arquitectura que asegure la escalabilidad para un sistema con los siguientes requisitos:

- a. Permitir al administrador la creación de aulas virtuales para escuelas primarias argentinas con sus correspondientes archivos multimedia: N videos incluyendo título y resumen
- b. Permitir a los alumnos el ingreso a cualquier aula y reproducción de video
- c. Permitir a los alumnos ver y crear comentarios sobre cualquiera de los videos
- d. Permitir ver un ranking de los videos más reproducidos con información actualizada cada hora como mínimo

Resuelva el problema mediante un diagrama de vista física y su explicación.

Referencias

-  → ejercicio crítico, la información no está en el resumen explícitamente, se *insinúa*
-  → hay que pensar, la información está en el resumen pero no detallada
-  → ejercicio repetido en otro coloquio
-  → hacer
-  → hacer con mucha prioridad
-  → lo dejamos por ahora, si tenemos tiempo lo hacemos